LESSON 28

算过的 f[i] 存起来

递推

学完以后，会从最小的开始一个个往上算，后面的用前面存好的结果。

第 20 课的爬楼梯，n 大了要等好一会儿

ways.cpp（第 20 课）

```
11      return ways(n - 1) + ways(n - 2);
```

一步走 1 级或 2 级，走上 n 级有几种走法。输入 45 试试，要等好一会儿。

ways(45) 要问 ways(44) 和 ways(43)，ways(44) 又要问 ways(43)……
同一个 ways(43) 被算了好几遍，越往下重复得越多。

算过的结果要是能存起来，就不用一遍遍重算了。

反过来：从最小的往上算，算好就存进数组

`f[1] = 1`	1 级：只有一种，走 1 级
`f[2] = 2`	2 级：1 + 1，或者直接 2
`f[3] = f[2] + f[1]`	最后一步走 1 级，前面有 f[2] 种；最后一步走 2 级，前面有 f[1] 种
`f[i-1] + f[i-2]`	f[i] 由前两个加出来

算 f[i] 的时候，f[i-1] 和 f[i-2] 早就存在数组里了，拿来就用。这叫**递推**。

f[1]、f[2] 先填好，for 往上推

stairs.cpp

```cpp
    int f[50] = {};
    f[1] = 1;
    f[2] = 2;
    for (int i = 3; i <= n; i++) {
        f[i] = f[i - 1] + f[i - 2];
    }
    cout << f[n] << endl;
```

前面先读入 n，n 是 1 到 45。

输入 5

8

起点：f[1] 和 f[2] 要先填好，后面全靠它们。

输入 45 也是一眨眼：每个 f[i] 只算一次。

小测：输入 4，输出几？

f[1] = 1, f[2] = 2, f[i] = f[i-1] + f[i-2]。

A. 1 4

B. 2 3

C. 3 5

D. 4 6

答对了！f[3] = 2 + 1 = 3，f[4] = 3 + 2 = 5。

输入 6：数组一格一格填满

起点	`f[1] = 1, f[2] = 2`	先填好
`i = 3、4`	`f[3] = 3, f[4] = 5`	`2 + 1, 3 + 2`
`i = 5、6`	`f[5] = 8, f[6] = 13`	`5 + 3, 8 + 5`
输出	`f[6]`	13

每一格只算一次，用的都是左边已经填好的格子。

猜一猜：输入 1

stairs.cpp

```
10      for (int i = 3; i <= n; i++) {
11          f[i] = f[i - 1] + f[i - 2];
12      }
13      cout << f[n] << endl;
```

屏幕上

1

i 从 3 开始，3 <= 1 不成立，for 一次也不进。会出错吗?

不会。f[1] 一开始就填好了，直接输出 1。
起点先填好，最小的输入也有答案。

动手改一改

从 stairs 开始。最后一步有几种走法？起点要填几个？

一步最多 3 级

一步可以走 1 级、2 级或 3 级。读入 n（1 到 30），输出走上 n 级有几种走法。3 级有 4 种：1+1+1、1+2、2+1、3。

输入 4	输入 10
7	**274**

Dev-C++：打开 starter/02_three_steps.cpp

杨辉三角：每个数等于肩上两个数的和

pascal.cpp

```
 8      for (int i = 0; i < n; i++) {
 9          c[i][0] = 1;
10          c[i][i] = 1;
11          for (int j = 1; j < i; j++) {
12              int left = c[i - 1][j - 1];
13              int right = c[i - 1][j];
14              c[i][j] = left + right;
15          }
```

输入 5

```
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
```

起点：每行头尾都是 1。left 是左肩，right 是右肩，都在上一行，所以一行一行往下推。

动手改一改

从 pascal 开始。走到一格，最后一步从哪来?

方格里有几条路?

m 行 n 列的方格（都是 1 到 10），从左上角走到右下角，每步只能往右或往下走一格。输出一共有几条路。

输入 3 3	输入 2 4	输入 1 5
6	**4**	**1**

Dev-C++: 打开 starter/04_grid.cpp

找错：起点错一个，后面全错

one_three.cpp（只阅读）

```cpp
    int f[50] = {};
    f[1] = 1;
    f[2] = 1;
    f[3] = 1;
    for (int i = 4; i <= n; i++) {
        f[i] = f[i - 1] + f[i - 3];
    }
```

点代码里的一行

一步走 1 级或 3 级，2 级只有 1+1 一种。

输入 4，应该是 3，屏幕上却是 2。

点你认为错了的那一行。

自己挑战

坏了一级台阶

一步走 1 级或 2 级，但第 k 级坏了，不能踩上去。读入 n 和 k（1 ⩽ k ⩽ n ⩽ 40），输出走上第 n 级有几种走法。

输入 5 3，输出

2

输入 5 1，输出

3

先自己写，卡住了再翻下一页。

卡住了？一次看一条提示

提示 1　踩不上去的那一级，走法就是 0：`f[k] = 0`。

提示 2　坏的可能是第 1 级或第 2 级，起点不好填。多放一个 `f[0] = 1`，当成站在地面上的那 1 种。

提示 3　`i` 从 1 推到 `n`：是 `k` 就填 0；`i` 是 1 就等于 `f[0]`；别的照样 `f[i-1] + f[i-2]`。

我的数列

自己定一条“后面由前面算出来”的规则，输出前 n 项。主题你自己定。

先做到

填好起点，用 for 往后推，输出前 n 项。

选你喜欢的

每天的金币、细胞分裂、存钱罐、植物长高，都可以。

还想更厉害

让一项用到前两项或前三项，或者让用户输入起点。

没有标准答案。写之前先手算前三项，再试试 n 是 1。

和同桌换着玩

1

换着运行

先听同桌说规则，自己手算前三项，再运行对一对

2

说一个喜欢的

告诉同桌，你最喜欢这个程序哪里

3

提一个想法

问问同桌：n 是 1 的时候对吗？

听完再改一版。想一想、做出来、试一试、分享、再改，程序就是这样越做越好的。

说得清吗?

Q1 **递推为什么比第 20 课的递归快?**

每个 f[i] 只算一次，存在数组里；递归会把同一个重复算很多遍。

Q2 **写递推要想清楚哪两件事?**

起点是什么，后一个怎么由前面的算出来。

Q3 **爬楼梯为什么是 f[i-1] + f[i-2]?**

最后一步要么走 1 级，要么走 2 级，两种情况加起来。

速查卡

`int f[50] = {};`	准备一个数组存结果
`f[1]、f[2]`	先填好起点
`i 从 3 到 n`	从起点后面一个开始往上推
`f[i-1] + f[i-2]`	`f[i]` 的规则：想清楚最后一步

这些后面再学

前缀和

也是递推：前 `i` 个的和，由前 `i-1` 个的和加一个数

`long long`

数太大 `int` 装不下，`n` 超过 `45` 时要换

动态规划

递推再往前走一步，以后学更难的题会遇到

今天学会递推：起点填好，后面的由前面算出来。

谢谢！

喜欢这节课？欢迎来 GitHub 和我交流。

 cpp-for-youth

回到课程首页

下一课“前缀和”还在准备中